

SafeSite AI

Maintained File Catalog

Edition	Day 3
Updated	2026-08-06
Maintained files	28
Current milestone	Video timing and frame sampling

This document explains the role, reason, and latest meaningful change for every maintained SafeSite AI project file. Generated caches, local secrets, Docker volumes, downloaded models, and temporary render files are intentionally excluded.

Maintenance protocol

Whenever a file is created, removed, or meaningfully changed, update the Markdown catalog and the generator metadata, regenerate this PDF, render every page, and include the documentation update in the same Git commit.

Root configuration

.gitattributes

Role: Marks generated PDFs, images, videos, and model weights as binary Git artifacts.

Why it exists: Prevents line-ending conversion, whitespace checks, and unreadable text diffs for binary files.

Latest change: Added binary rules for PDF, image, video, and PyTorch model artifacts.

.env.example

Role: Documents the environment variables required by PostgreSQL and FastAPI.

Why it exists: Lets developers create a local .env file without committing real credentials.

Latest change: Initial database name, user, password, and connection URL added.

.gitignore

Role: Excludes local, generated, cached, secret, and heavy files from Git.

Why it exists: Keeps commits focused on maintained source files.

Latest change: Initial Python, editor, data, model, MLflow, and temporary-file exclusions added.

README.md

Role: Provides the project overview, startup commands, learning links, and working rules.

Why it exists: Acts as the main entry point for every developer and reviewer.

Latest change: Added the Day 3 video lesson and reproducible ingestion exercise commands.

compose.yaml

Role: Defines and connects the PostgreSQL and FastAPI containers.

Why it exists: Starts the current system reproducibly and preserves database data in a volume.

Latest change: Added the optional ingestion tools profile with a bind-mounted data workspace.

Database

database/init/001_schema.sql

Role: Creates the violations table, constraints, indexes, and fake seed events.

Why it exists: Establishes the structured event contract inside PostgreSQL.

Latest change: Initial camera, track, violation, confidence, evidence, and timestamp fields added.

API container

services/api/Dockerfile

Role: Builds the container image that runs the FastAPI application.

Why it exists: Packages Python, dependencies, source code, and the startup command reproducibly.

Latest change: Split the image into reusable base, runtime, and test build stages.

services/api/requirements.txt

Role: Declares the Python dependencies required by the API.

Why it exists: Makes dependency installation repeatable in the container.

Latest change: FastAPI, Uvicorn, Pycpg, pool, and settings dependencies added.

services/api/requirements-dev.txt

Role: Declares dependencies used only by automated API tests.

Why it exists: Keeps Pytest and HTTPX separate from the production runtime image.

Latest change: Added Pytest and HTTPX for black-box integration testing.

API application

services/api/app/__init__.py

Role: Marks the app directory as a Python package.

Why it exists: Makes imports such as app.main and app.schemas predictable.

Latest change: Initial package marker added.

services/api/app/config.py

Role: Loads the PostgreSQL connection URL from environment configuration.

Why it exists: Separates deployment configuration from application logic.

Latest change: Initial Pydantic settings model and development default added.

services/api/app/database.py

Role: Creates and manages the asynchronous PostgreSQL connection pool.

Why it exists: Reuses connections efficiently and manages them with the API lifecycle.

Latest change: Initial dictionary-row pool, startup wait, and shutdown functions added.

services/api/app/schemas.py

Role: Defines Pydantic request and response contracts for violations and statistics.

Why it exists: Validates JSON fields, types, ranges, and allowed violation values.

Latest change: Initial creation, response, count, summary, and violation-type models added.

services/api/app/main.py

Role: Defines the FastAPI application lifecycle and HTTP routes.

Why it exists: Exposes health, create, list, filter, and summary operations over PostgreSQL.

Latest change: Added inclusive occurrence-time filters and reversed-range validation.

API tests

services/api/tests/test_time_filters.py

Role: Tests time filtering, invalid ranges, and SQL-injection resistance against the running stack.

Why it exists: Proves that HTTP, validation, SQL, and PostgreSQL work correctly together.

Latest change: Added three integration tests with automatic test-data cleanup.

Ingestion container

services/ingestion/Dockerfile

Role: Builds the reproducible OpenCV environment for video-ingestion tools.

Why it exists: Gives every developer the same Python, decoder, and image-processing runtime.

Latest change: Added the Python 3.12 image, OpenCV installation, application copy, and help command.

services/ingestion/requirements.txt

Role: Declares the ingestion service's OpenCV dependency.

Why it exists: Makes video decoding, generation, and image writing reproducible inside Docker.

Latest change: Added the supported OpenCV headless version range.

Ingestion application

services/ingestion/app/__init__.py

Role: Marks the ingestion app directory as a Python package.

Why it exists: Allows tools to run consistently with `python -m app..`

Latest change: Initial package marker added.

services/ingestion/app/generate_demo_video.py

Role: Generates a controlled MP4 with known timing, resolution, and frame count.

Why it exists: Provides deterministic input without relying on personal or downloaded footage.

Latest change: Added configurable generation and a moving PPE-equipped worker scene.

services/ingestion/app/sample_frames.py

Role: Samples video frames by time and writes JPEG evidence plus JSONL metadata.

Why it exists: Establishes the ingestion contract later consumed by Kafka, MinIO, and YOLO.

Latest change: Added validation, time-based sampling, traceable names, and metadata records.

Learning documentation

docs/ARCHITECTURE.md

Role: Explains the architecture by following one PPE violation event.

Why it exists: Builds a mental model connecting every planned system layer.

Latest change: Initial current and weekly architecture stages documented.

docs/LESSON_01.md

Role: Contains the first guided lesson and practical exercises.

Why it exists: Teaches Docker, networking, validation, persistence, and event flow.

Latest change: Initial Day 1 exercises and explanation checkpoint added.

docs/LESSON_02.md

Role: Contains the second guided lesson and implementation exercise.

Why it exists: Teaches HTTP, async waiting, pooling, safe SQL, time filters, and tests.

Latest change: Initial Day 2 lesson and explanation checkpoint added.

docs/LESSON_03.md

Role: Contains the video-fundamentals lesson and frame-sampling exercise.

Why it exists: Teaches FPS, resolution, codecs, timestamps, sampling, and JSONL metadata.

Latest change: Added the Day 3 lesson, Docker commands, contract, and checkpoint.

docs/ROADMAP.md

Role: Defines the realistic 28-day learning and implementation plan.

Why it exists: Protects the deadline using must-have, should-have, and stretch priorities.

Latest change: Initial plan adapted for a GTX 1650 with 4 GB VRAM.

docs/FILE_CATALOG.md

Role: Provides the human-readable source catalog for maintained project files.

Why it exists: Keeps file responsibilities and meaningful changes understandable.

Latest change: Added all Day 3 ingestion files and recorded video generation and sampling changes.

Documentation tooling and output

scripts/generate_file_catalog_pdf.py

Role: Generates this polished catalog PDF from maintained metadata.

Why it exists: Makes future PDF updates consistent and repeatable.

Latest change: Added Day 3 ingestion entries, milestone metadata, and regenerated pagination.

output/pdf/SafeSite_AI_File_Catalog.pdf

Role: Provides the printable and shareable form of the file catalog.

Why it exists: Supports project reviews, learning, and PFE interview preparation.

Latest change: Regenerated after the Day 3 ingestion service, tools, README, Compose, and lesson changes.

Change register

2026-08-06 - Video timing and frame sampling

Added a reproducible OpenCV ingestion container, controlled demo-video generation, time-based JPEG sampling, JSONL metadata, fractional-FPS validation, and the Day 3 lesson.

2026-08-05 - Safe time filtering and tests

Added inclusive occurrence-time filters, reversed-range validation, a multi-stage test image, black-box integration tests, the Day 2 lesson, and binary Git attributes.

2026-08-04 - Initial vertical slice

Added Docker Compose, PostgreSQL schema and seed data, FastAPI routes, architecture explanation, Day 1 lesson, and the 28-day roadmap.